

Since there is no corresponding data type in Python for an Erlang token, `py_interface` contains a function to convert a string to an Erlang token. In the code below, the Python client signs on with the name 'python'. The 'mb.`Self()`' function returns the process ID. The Erlang server is running on the node 'messenger@netbook' with the registered name 'messenger'.

```
def logon_callback():
    global mb
    msg=(mb.Self(), erl_term.ErlAtom('logon'), erl_term.
ErlAtom('python'))
    mb.Send(('messenger', 'messenger@netbook'), msg)
```

A message you send to the messenger for another user will be a tuple with the following four elements:

- Sender's ID
- Erlang token 'message_to'
- The name of the person for whom the message is intended
- Content of the message

However, the format of the message you receive will be different, as you can see from the Erlang code. Server transfer validates the sender, and then it validates the recipient and sends a message to the recipient and a confirmation message to the sender:

```
server_transfer(From, To, Message, User_List) ->
case lists:keysearch(From, 1, User_List) of
false ->
    From ! {messenger, stop, you_are_not_logged_on};
    {value, {From, Name}} ->
        server_transfer(From, Name, To, Message, User_List)
end.
```

```
server_transfer(From, Name, To, Message, User_List) ->
case lists:keysearch(To, 2, User_List) of
false ->
```

```
From ! {messenger, receiver_not_found};
{value, {ToPid, To}} ->
    ToPid ! {message_from, Name, Message},
    From ! {messenger, sent}
end.
```

The message you receive will be a tuple with the following three elements:

- Sender's ID
- Sender's name
- The contents of the message

The message will be received by the `MBoxCallback`. In the simple example given below, you receive the message and send a canned message back to the sender:

```
def MBoxCallback(msg, *k, **kw):
    global mb
    if len(msg) > 2:
        # send back a vacation message to the sender
        m = (mb.Self(), erl_term.ErlAtom('message_to'),
msg[1], "on vacation")
        mb.Send(('messenger', 'messenger@netbook'), m)
    else:
        print "Incoming msg=%s (k=%s, kw=%s)" % ('msg', 'k',
'kw')
```

You can learn more by looking at the examples included with the `py_interface` distribution. The usual doc string documentation is available in the `py_interface` modules.

So, in case a part of your application could make use of parallel processing, you could integrate your existing Python (or C or Java) application with Erlang. It is well worth your time to explore this option. 

By: Anil Seth

The author has earned the right to do what interests him. You can find him online at <http://sethanil.com>, <http://sethanil.blogspot.com>, and reach him via email at anil@sethanil.com

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

Your favourite magazine on
Open Source is now on the Web, too.

LinuxForU.com

Follow us on Twitter@LinuxForU

